

FLEETMANAGER DESIGN

1 INTRODUZIONE

Il presente documento descrive il design del sistema FleetManager, con particolare riferimento alla modellazione UML, all'architettura software e alle principali decisioni progettuali adottate durante lo sviluppo. L'obiettivo del documento è fornire una visione strutturata e coerente delle scelte di progettazione, mostrando come i requisiti funzionali e non funzionali siano stati tradotti in modelli e soluzioni architetturali concrete.

Il design del sistema è stato sviluppato a partire dal documento di Analisi dei Requisiti e costituisce un elemento di collegamento tra la fase di analisi e l'implementazione. I diagrammi UML sono utilizzati come strumento principale per rappresentare sia la struttura statica del sistema sia il suo comportamento dinamico, supportando la comprensione del dominio applicativo e delle interazioni tra le componenti.

Il documento include diagrammi UML già realizzati, diagrammi attualmente in fase di definizione e diagrammi pianificati ma non ancora sviluppati. Per questi ultimi viene fornita una descrizione del contesto e della situazione che si intende rappresentare, al fine di mantenere una struttura completa e coerente del documento di design, anche in presenza di artefatti non ancora finalizzati.

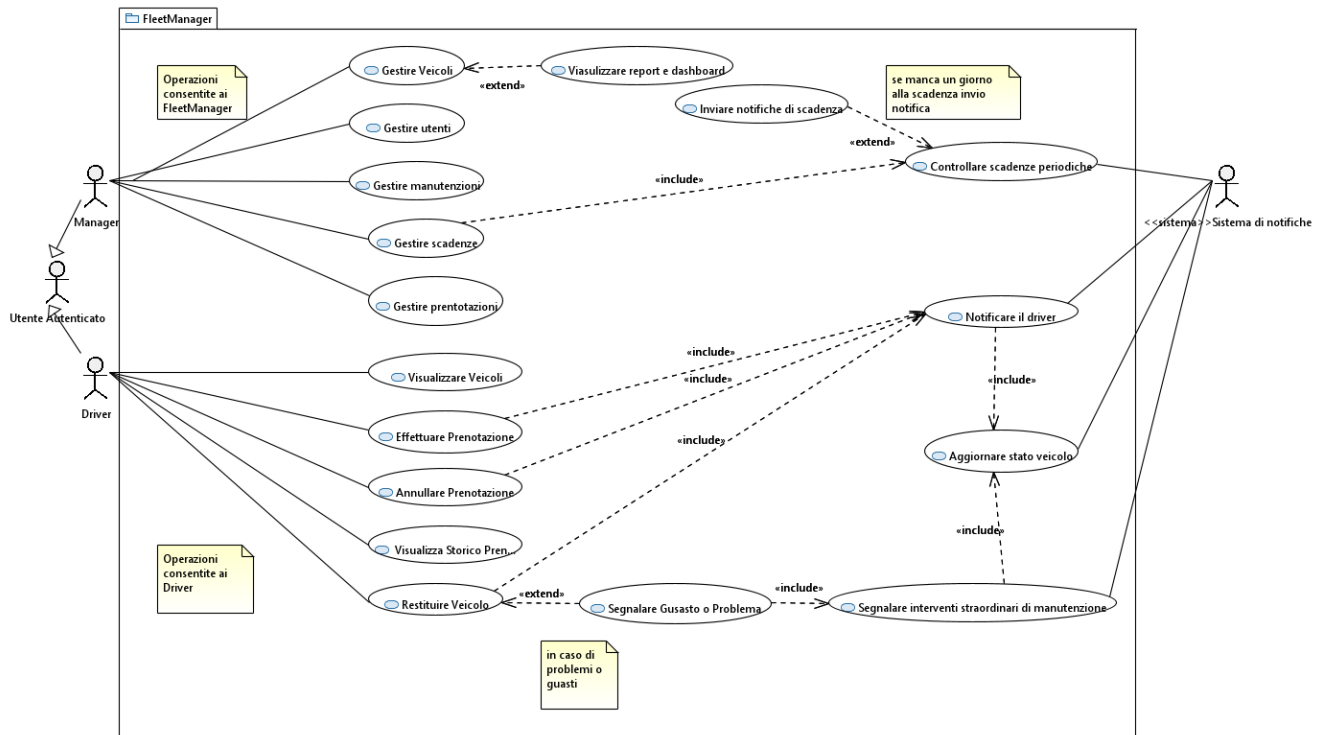
Il documento è organizzato come segue:

- la sezione Modelling descrive i principali diagrammi UML utilizzati o pianificati nel progetto, specificandone scopo, contesto e stato di avanzamento;
- la sezione Software Architecture presenta l'architettura del sistema e l'organizzazione ad alto livello delle componenti;
- la sezione Software Design illustra i principi progettuali e i pattern adottati, evidenziando la coerenza tra modelli UML e codice;
- la sezione finale riassume le principali considerazioni sul design complessivo del sistema.

Questa impostazione consente di mantenere una visione unitaria del progetto, evidenziando il ruolo del design come elemento centrale per garantire chiarezza, manutenibilità ed evoluzione del sistema FleetManager.

2. Modelling (UML)

2.1 Use Case Diagram



Scopo

Il Use Case Diagram ha lo scopo di rappresentare le **funzionalità offerte dal sistema FleetManager** dal punto di vista degli attori esterni, fornendo una visione ad alto livello dei servizi messi a disposizione e dei confini del sistema. Questo diagramma consente di collegare in modo diretto i requisiti funzionali definiti nell'Analisi dei Requisiti con le funzionalità effettivamente progettate.

Contesto e contenuto

Il diagramma modella le principali interazioni tra il sistema e gli attori coinvolti nella gestione della flotta. In particolare, rappresenta le operazioni fondamentali legate alla gestione dei veicoli, delle prenotazioni, delle scadenze e delle attività di manutenzione.

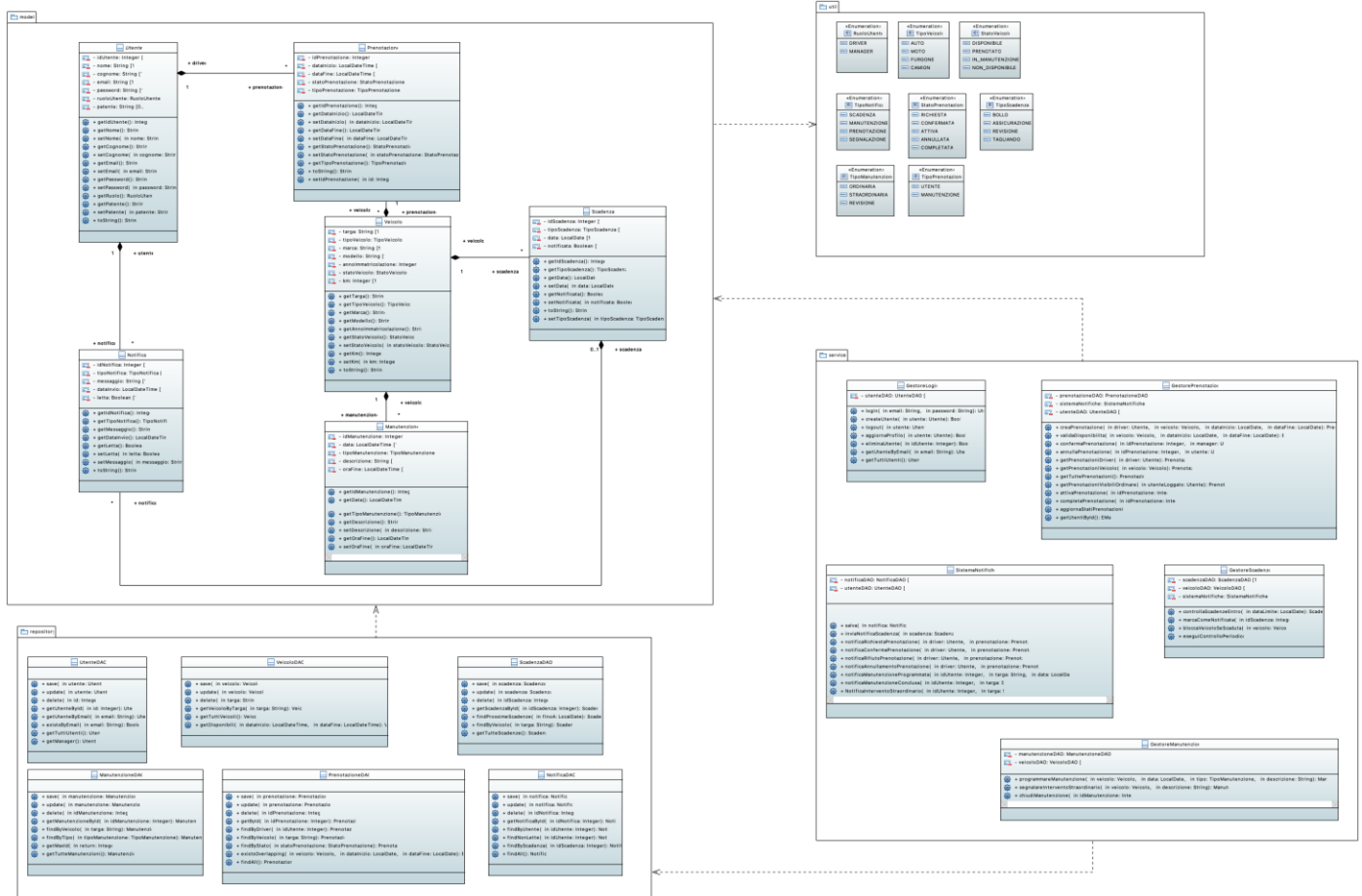
Il Use Case Diagram è utilizzato come riferimento iniziale per comprendere **cosa il sistema deve fare**, senza entrare nei dettagli di come tali funzionalità vengano implementate internamente.

Il diagramma fornisce inoltre una base comune per gli altri modelli UML, in particolare per il Class Diagram e i diagrammi dinamici, che approfondiscono rispettivamente la struttura del sistema e il comportamento associato ai singoli casi d'uso.

Stato del diagramma

Il Use Case Diagram è **realizzato** ed è stato modellato utilizzando lo strumento Papyrus. Esso riflette i requisiti funzionali attualmente definiti e costituisce il punto di partenza per le successive attività di progettazione e implementazione.

2.2 Class Diagram



Scopo

Il Class Diagram ha lo scopo di rappresentare la struttura statica del sistema FleetManager, descrivendo le principali classi di dominio, i loro attributi, le operazioni esposte e le relazioni che intercorrono tra di esse.

Questo diagramma costituisce il modello centrale del design del sistema, in quanto definisce le entità fondamentali del dominio applicativo e funge da riferimento diretto per l'implementazione in Java.

Contesto e contenuto

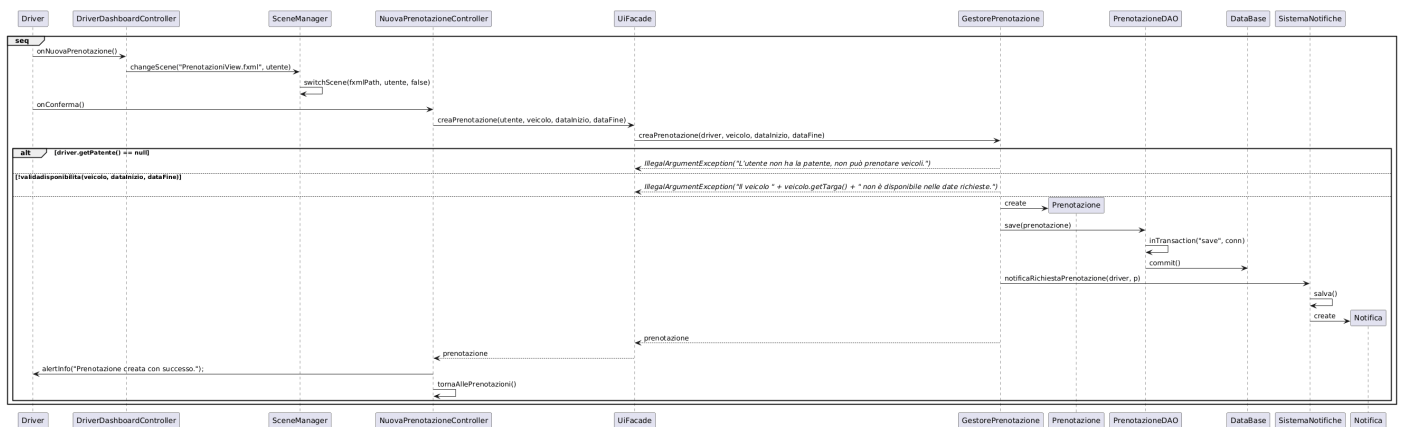
Il diagramma modella le principali entità del dominio FleetManager, tra cui utenti, veicoli, prenotazioni, manutenzioni, scadenze e notifiche.

Vengono rappresentate le associazioni tra le classi, l'uso di enumerazioni per descrivere stati e tipologie e la separazione concettuale tra le diverse responsabilità del sistema.

Stato del diagramma

Il Class Diagram è stato realizzato utilizzando **Papyrus**. A partire da esso è stata generata una prima versione del codice, successivamente estesa e rifinita manualmente.
Il diagramma viene mantenuto coerente con l'implementazione ed è aggiornato in modo iterativo durante lo sviluppo del sistema.

2.3.1 Sequence Diagram – Creazione Prenotazione (Driver)



Scopo

Il Sequence Diagram ha lo scopo di descrivere il flusso temporale delle interazioni che avvengono durante la creazione di una prenotazione da parte di un driver.

Contesto e contenuto

Il diagramma rappresenta il flusso completo che parte dall'interazione dell'utente con l'interfaccia grafica e prosegue attraverso:

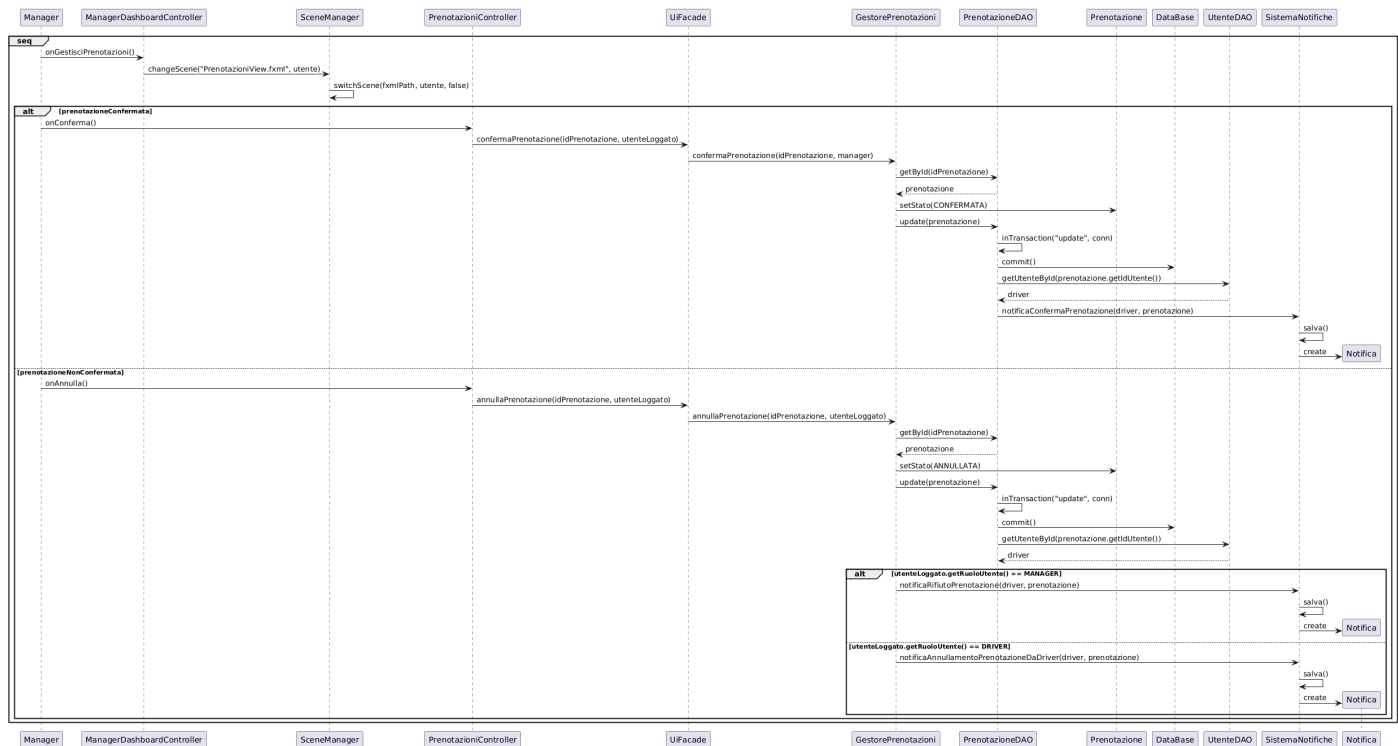
- cambio scena tramite SceneManager;
- invocazione della UIFacade;
- logica applicativa nel GestorePrenotazioni;
- persistenza della prenotazione tramite PrenotazioneDAO;
- generazione della notifica associata.

Il diagramma include strutture di controllo alternative per la gestione dei casi di errore, come la mancanza di patente o la non disponibilità del veicolo.

Stato del diagramma

Il Sequence Diagram è stato realizzato utilizzando **PlantUML** ed è coerente con l'implementazione effettiva del flusso di prenotazione nel codice Java.

2.3.2 Sequence Diagram – Gestione Prenotazione (Manager)



Scopo

Questo Sequence Diagram ha lo scopo di descrivere il comportamento del sistema durante la gestione di una prenotazione da parte del manager, in particolare le operazioni di conferma o annullamento.

Contesto e contenuto

Il diagramma rappresenta:

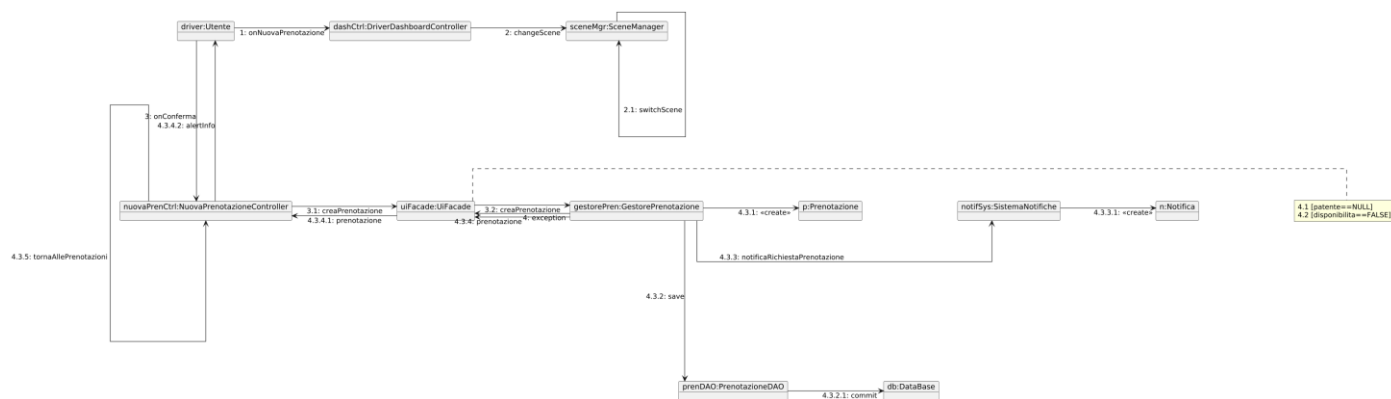
- l'accesso del manager alla schermata di gestione prenotazioni;
- la conferma o l'annullamento della prenotazione;
- l'aggiornamento dello stato della prenotazione;
- la persistenza delle modifiche;
- la generazione delle notifiche verso il driver.

Sono presenti strutture alternative che distinguono i flussi in base allo stato della prenotazione e al ruolo dell'utente che effettua l'operazione.

Stato del diagramma

Il Sequence Diagram è stato realizzato utilizzando **PlantUML** ed è allineato con la logica di business implementata nel service layer.

2.4 Communication Diagram



Scopo

Il Communication Diagram ha lo scopo di rappresentare le interazioni tra gli oggetti del sistema FleetManager, enfatizzando le relazioni strutturali e i collegamenti tra le istanze coinvolte in uno scenario applicativo.

Contesto e contenuto

Il diagramma modella uno scenario di creazione di una prenotazione da parte di un driver, mostrando la collaborazione tra:

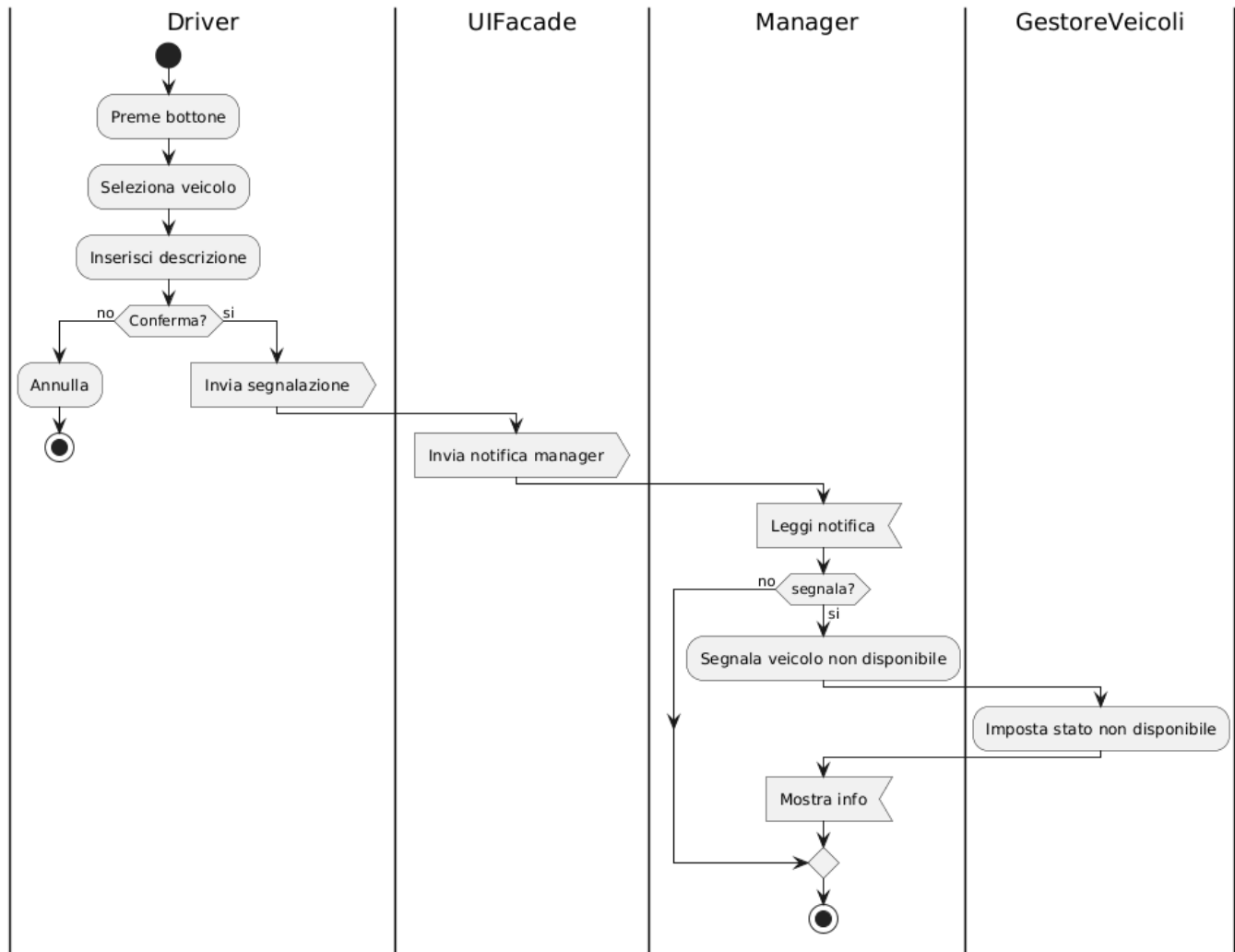
- controller dell'interfaccia grafica;
- SceneManager;
- UiFacade;
- gestori applicativi;
- DAO e database;
- sistema di notifiche.

Il diagramma mette in evidenza il ruolo centrale della facade nel coordinare le interazioni tra UI e service layer, riducendo l'accoppiamento diretto tra i componenti.

Stato del diagramma

Il Communication Diagram è stato realizzato utilizzando **PlantUML** ed è utilizzato come complemento al Sequence Diagram, fornendo una vista alternativa focalizzata sulle relazioni tra oggetti.

2.5 Activity Diagram



Scopo

L'Activity Diagram ha lo scopo di rappresentare il flusso di attività associato a un processo del sistema FleetManager, evidenziando azioni, decisioni e possibili diramazioni.

Contesto e contenuto

Il diagramma modella un processo applicativo che coinvolge più attori e componenti, mostrando:

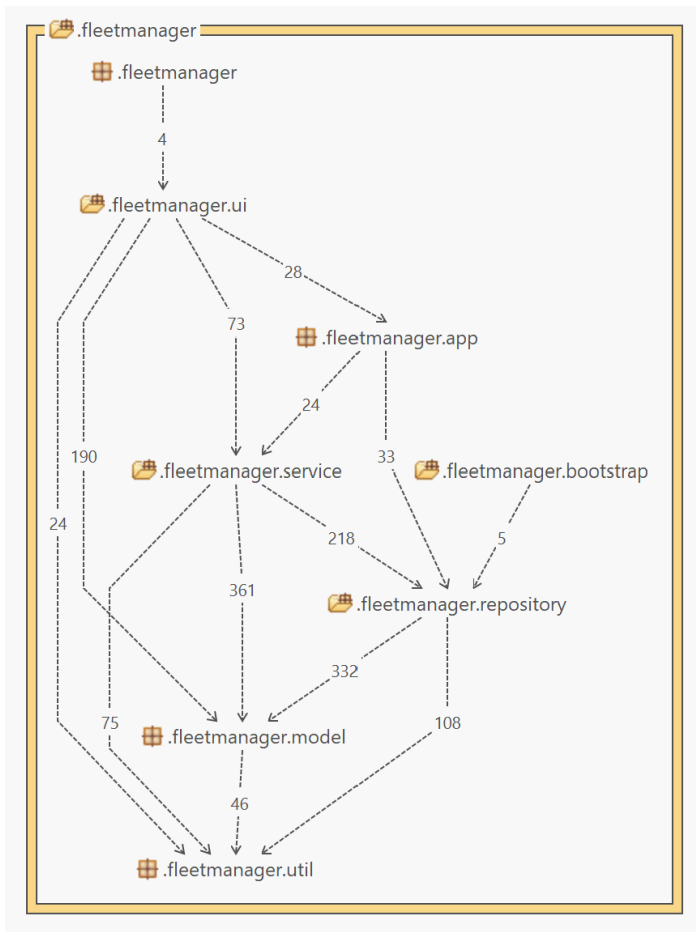
- le azioni eseguite dall'utente;
- le decisioni che influenzano il flusso;
- il coordinamento tra UI e logica applicativa.

L'Activity Diagram fornisce una visione globale del processo, complementare ai diagrammi di sequenza.

Stato del diagramma

L'Activity Diagram è stato realizzato utilizzando **PlantUML** ed è completo per lo scenario analizzato.

2.6 Package Diagram



Scopo

Il Package Diagram ha lo scopo di rappresentare l'organizzazione strutturale del sistema FleetManager in termini di package e delle dipendenze esistenti tra di essi.

Questo diagramma consente di analizzare la modularità del sistema e di verificare il rispetto dei principi di separazione delle responsabilità e architettura a livelli.

Contesto e contenuto

Il diagramma mostra la suddivisione del progetto nei principali package applicativi:

- it.fleetmanager.ui, contenente i controller JavaFX e le viste;
- it.fleetmanager.service, che rappresenta il service layer e la logica applicativa;
- it.fleetmanager.repository, che incapsula l'accesso ai dati tramite DAO e database;
- it.fleetmanager.model, che include le classi di dominio;
- it.fleetmanager.util, che raccoglie classi di supporto ed enumerazioni;
- it.fleetmanager.bootstrap, responsabile dell'inizializzazione del sistema.

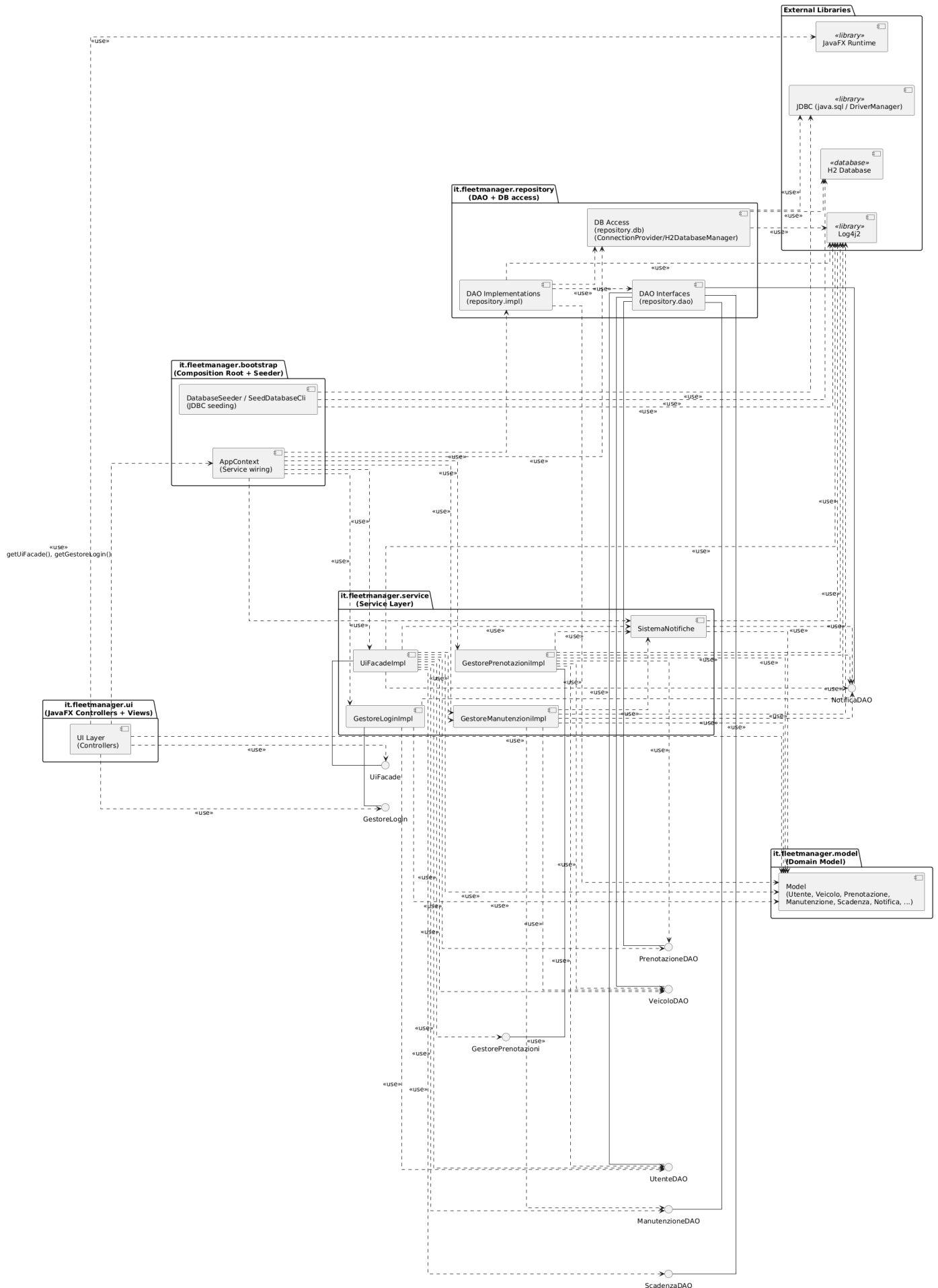
Le dipendenze tra i package evidenziano un'architettura stratificata, in cui i livelli superiori dipendono solo dai livelli inferiori, evitando dipendenze cicliche dirette tra UI e repository.

Stato del diagramma

Il diagramma è utilizzato come strumento di verifica architeturale e supporta le attività di refactoring e controllo della qualità del codice.

2.7 Component Diagram

FleetManager - Component Diagram



Scopo

Il Component Diagram ha lo scopo di descrivere l'architettura software del sistema FleetManager, mostrando le principali componenti, le loro responsabilità e le relazioni di dipendenza tra di esse. Il diagramma fornisce una visione ad alto livello dell'organizzazione del sistema e dei confini tra i diversi sottosistemi.

Contesto e contenuto

Il diagramma rappresenta le seguenti componenti principali:

- **UI Layer**, contenente i controller JavaFX e le viste;
- **Service Layer**, che include i gestori applicativi e la facade;
- **Repository Layer**, composto da interfacce DAO, implementazioni e accesso al database;
- **Model**, che rappresenta il dominio applicativo;
- **Bootstrap**, responsabile del wiring delle dipendenze e del seeding iniziale del database.

Sono inoltre evidenziate le dipendenze verso librerie esterne, come JavaFX, JDBC, Log4j2 e il database H2 embedded.

Stato del diagramma

Il Component Diagram è stato realizzato utilizzando **PlantUML** ed è coerente con l'architettura multilayer implementata nel sistema.

Il diagramma viene utilizzato come riferimento principale nella sezione di Software Architecture.